SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

Faculty of Computing Department of Software Engineering

SE3090 Model Paper Marking Scheme 2026

Software Engineering Frameworks Year 3 Semester 1

Model answers Partial marks Student self assessment

Use this guide after attempting the 100-mark model paper. It explains what earns each mark and gives example answers. Equivalent answers can earn the same marks when they satisfy the stated requirement; the examples are not the only acceptable wording.

How to mark and improve your answers

For short explanations, name the concept and explain its effect in this scenario. A term without the requested reason usually earns only part of the mark.

For a justified choice, link the choice to a stated requirement or constraint. Add a trade-off where the question asks for one.

For code blanks, complete only the numbered fragment so it fits the surrounding code. Award independent marks and avoid penalizing a single mistake repeatedly.

For code-writing tasks, use the rubric to check behaviour, data handling and failure paths. Equivalent Fetch/Axios or framework solutions are accepted where stated.

Do not exceed a question's maximum. Do not add negative marks to structured questions. Retain MCQ fractions until calculating the overall score.

Marks by section

Section	Focus	Marks
A	Multiple choice questions	15
B	Backend development with ASP.NET Core and PostgreSQL	30
C	Frontend development with React	15
D	Mobile development with Flutter	15
E	Hosting selection and engineering practice	10
F	Agentic software development	15
Total	All 30 questions	100

MCQ scoring for this model

Single answer: award 1 only for the correct sole selection; otherwise 0. Multiple answers: score = max(0, (correct selections - wrong selections) / number of correct options). The maximum is 1 mark per question.

Example: Question 6 has two correct options. One correct alone earns 0.5; one correct and one wrong earns 0; both correct alone earn 1. This rule applies to this practice paper.

Section A Answer key and reasons

Q	Answer	Type	Reason
1	b	Single	Serverless functions suit event-driven work such as image processing. The provider manages the function execution infrastructure.
2	a,d,e	Multiple	Use team capability, a requirements-based prototype, licensing and integration constraints. Popularity alone does not establish project fit.
3	d	Single	useMemo caches a computed value using its dependency list; useCallback caches a function reference.
4	e	Single	var uses compile-time type inference. It does not remove static typing or permit a later assignment of an incompatible type.
5	c	Single	EF Core migrations track changes to the database schema as the application model evolves.
6	b,d	Multiple	Use related-data loading deliberately and check the actual generated queries. An ORM does not guarantee good query performance.
7	a	Single	A soft reset moves the branch back while retaining the changes in the staging area. The question explicitly concerns an unpushed local commit.
8	d	Single	Expanded gives its child the remaining available space in a Row or Column.
9	b	Single	The null-coalescing operator ?? supplies its right-hand value when the left-hand expression is null.
10	c	Single	The backend CORS policy controls which browser origins may access the response. It does not replace API authentication or authorization.
11	a,c,e	Multiple	Hooks need a stable call order. Effects should have appropriate dependencies and clean up external resources.
12	b,d,e	Multiple	UNIQUE prevents duplicates, NOT NULL rejects NULL and a foreign key enforces the referenced row. NOT NULL alone does not reject an empty string.
13	a,c,e	Multiple	Source analysis, dependency checks and secret scanning address different risks. Formatting and screenshots are not substitutes for these checks.
14	a	Single	Widget tests exercise a widget or screen and its user interactions in isolation.
15	a,d	Multiple	Tracing exposes the execution path for diagnosis and performance analysis. Correctness still requires evaluation.

Each item carries 1 mark. Apply the practice scoring rule on the first page.

Section B Backend development with ASP NET Core and PostgreSQL

Section total 30 marks

Question 16 API contracts and testable design (6 marks)

16.1 Input contract 2 marks

Model answer: the reporter supplies Title, RoomId and Priority. Id is generated by the database and is not chosen by the reporter. InternalStaffNote is an internal staff-only field; accepting it would let a reporter set data outside the intended input contract. Status is initialized by the server.

Mark allocation: Lists Title, RoomId and Priority, one third of a mark for each (1); Explains that Id is server/database generated (0.5); Explains the internal-field or over-posting risk of accepting InternalStaffNote (0.5).

16.2 Interface and test double 2 marks

Model answer: inject a fake or mock ITicketService that returns a known ticket result instead of a production service. The test then checks how the controller uses the result without real database or network dependencies, making the test isolated and repeatable.

Mark allocation: A fake/mock implementing ITicketService is supplied through the constructor (1); A specific test benefit such as isolation, predictable results or avoiding a real database (1).

16.3 Asynchronous database work 2 marks

Model answer: when the database Task is incomplete, await suspends the async method and allows the request thread to do other work until the operation completes. Blocking with .Result ties up a thread while waiting; awaiting I/O helps the server handle concurrent requests without using one blocked thread for every wait.

Mark allocation: Correctly explains suspension/non-blocking waiting for an incomplete Task (1); Connects this to avoiding a blocked thread and handling concurrent requests (1).

Answer technique: A phrase such as "async is faster" is too vague for the second mark. Explain the effect on waiting and request threads.

Revision sources: L03 PDF pp19,26,39,41; Lab03 Tasks02-03.

Question 17 Database constraints and schema changes (6 marks)

17.1 SQL completions 4 marks

Blank	Required completion	Marks
1	REFERENCES	1
2	NOT NULL	1
3	CHECK	1
4	'Open'	1

Use a quoted SQL string literal for [4]. NOT NULL rejects NULL; DTO validation also checks blank input.

17.2 Migration commands 2 marks

```
dotnet ef migrations add AddTickets
dotnet ef database update
```

Mark allocation: Creates the AddTickets migration (1); Then applies migrations using database update (1).

Valid project/startup-project command options are accepted.

Revision sources: L04 PDF pp25-26,32; Lab04 Task02.

Question 18 Validation filtering and persistence (12 marks)

Each correctly completed blank earns 1 mark. Award independently; do not repeat a penalty across later correct answers. The completion must fit the displayed code.

18.1 Validation 3 marks

Blank	Required completion	Marks
1	Required	1
2	StringLength(120)	1
3	Range(1, 5)	1

Accept MaxLength(120) for [2]. The Required attribute separately rejects missing, empty or whitespace-only Title values under normal validation.

18.2 Filtered projected query 5 marks

Blank	Required completion	Marks
4	Where	1
5	t.Status == "Open"	1
6	OrderByDescending	1
7	Select	1
8	ToListAsync	1

[5] may use an equivalent exact comparison. Ordering only by room or title does not meet the priority requirement. ToList without asynchronous execution does not meet [8].

18.3 Persistence 4 marks

Blank	Required completion	Marks
9	dto.Title	1
10	"Open"	1
11	Add	1
12	SaveChangesAsync	1

Answer technique: Add marks the entity for insertion; SaveChangesAsync persists it. The generated id is returned after the save. Do not write controller code for this question.

Revision sources: L03 PDF p50; L04 PDF pp31-33,35; Lab03 Task02; Lab04 Task02.

Question 19 Permission policies and token validation (6 marks)

19.1 Policy completions 2 marks

Blank	Required completion	Marks
1	RequireClaim	1
2	Policy	1

```
options.AddPolicy("AssignTickets", policy =>
    policy.RequireClaim("perm", "tickets.assign"));
[Authorize(Policy = "AssignTickets")]
```

19.2 Permission failure 2 marks

Model answer: return 403 Forbidden. The caller is authenticated, but this policy requires a perm claim with the specific value tickets.assign; the Technician role alone does not supply that permission.

Mark allocation: 403 Forbidden for an authenticated policy failure (1); Explains the required claim/value and why the role is insufficient (1).

19.3 Token issuer and audience 2 marks

Model answer: validate the issuer against the trusted issuer configured for CampusCare, so a token from another organization is not accepted. Validate the audience against the identifier expected by this API, so a token intended for another application cannot be reused here.

Mark allocation: Issuer validation and its purpose (1); Audience validation and its purpose (1).

For each check, award 0.5 for naming it and 0.5 for its purpose. Merely repeating signature and expiry does not answer this part.

Revision sources: L04 PDF p51; Lab04 Tasks04-05.

Section C Frontend development with React

Section total 15 marks

Question 20 Route loading and component identity (4 marks)

20.1 On demand loading 2 marks

Model answer: use React.lazy (or lazy imported from React) with Suspense. lazy loads the analytics component through a dynamic import; Suspense shows a fallback while it loads. Users who do not open analytics download less code initially.

Mark allocation: React.lazy / lazy (0.5); Suspense as the loading boundary/fallback (0.5); A specific initial-bundle or initial-load benefit (1).

20.2 Stable row identity 2 marks

Model answer: an index identifies the current position, so deleting or sorting rows changes which ticket occupies that position and can preserve a child's local input state against the wrong ticket. Use key={ticket.id} (or the equivalent variable name) to give each row a stable identity.

Mark allocation: Explains position changes and the resulting component/state identity problem (1); Correct stable unique id key expression (1).

Revision sources: L02 PDF pp29,37,44; Lab02 Task02.

Question 21 Submitting a report from React (6 marks)

21.1 Handler body 4 marks

```
e.preventDefault();
const cleanTitle = title.trim();
if (!cleanTitle) {
    setError('Enter a title');
    return;
}
setSaving(true);
setError(null);
try {
    const response = await fetch(
        `${import.meta.env.VITE_API_URL}/api/tickets`,
        {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                title: cleanTitle, roomId, priority
            })
        }
    );
    if (!response.ok) {
        throw new Error(`Submit failed: ${response.status}`);
    }
    onCreated(await response.json());
} catch (err) {
    setError(err instanceof Error
        ? err.message : 'Could not submit the report');
} finally {
    setSaving(false);
}
```

Mark allocation: Prevents default submission (0.5); trims/validates title, stores an error and returns before requesting if blank (0.5) (1); Uses the correct URL and POST method (0.5); sends the required JSON fields with the JSON content type (0.5) (1); Checks response success before using JSON (0.5); passes the created JSON to onCreated (0.5) (1); Sets saving and clears the old error before requesting (0.25); catches/stores an error (0.25); resets saving on success and failure (0.5) (1).

Accept equivalent Axios code with equivalent error handling. Accept a specific status check for 201, as the endpoint contract specifies 201. Clearing the title after success is optional and earns no extra marks.

21.2 Submission feedback 2 marks

```
<button type="submit" disabled={saving}>
    {saving ? 'Submitting...' : 'Submit'}
</button>
{error && <p role="alert">{error}</p>}
```

Mark allocation: A submit button disabled while saving (0.5), with a label that changes during submission (0.5) (1); An error message displayed only when error is present (1).

Answer technique: Answer exactly the requested handler body and two fragments. A full component is unnecessary. Frontend validation helps the user; the backend still validates input.

Revision sources: L02 PDF pp26,29,32; Lab02 Tasks04-05.

Question 22 Cache identity after a mutation (5 marks)

22.1 Mutation and invalidation 3 marks

Blank	Required completion	Marks
1	useQueryClient	1
2	useMutation	1
3	invalidateQueries	1

```
const queryClient = useQueryClient();
const addTicket = useMutation({
    mutationFn: createTicket,
    onSuccess: () => queryClient.invalidateQueries({
        queryKey: ['tickets', roomId]
    })
});
```

22.2 Key and freshness 2 marks

Model answer: roomId distinguishes the cached list for one room from another room's list, preventing them from sharing the same cache identity. After creation, invalidation marks the matching cached query stale and triggers refetching for active matching queries, so the displayed list can include the new ticket.

Mark allocation: Room-specific cache identity and its consequence (1); Cached data becomes stale after creation and should be refreshed (1).

Do not award the second mark for claiming that a create operation automatically inserts the new ticket into every cache without invalidation or an explicit cache update.

Revision sources: L02 PDF p34.

Section D Mobile development with Flutter

Section total 15 marks

Question 23 Mobile layers and a camera feature (5 marks)

23.1 Mobile responsibilities 3 marks

Model answer: the UI layer builds the form, displays status and handles taps. The service/logic layer decides whether a report is ready and coordinates submission. The repository/data layer hides HTTP/local-storage access and gets or saves ticket data.

Mark allocation: Scenario-linked UI responsibility (1); Scenario-linked service or logic responsibility (1); Scenario-linked repository or data responsibility (1).

23.2 Camera choice and fallback 2 marks

Model answer: check that the plugin supports both Android and iOS and review the required platform setup/permissions. If camera access fails or is denied, let the reporter submit the title and room without a photograph and explain that the photo is optional.

Mark allocation: A concrete Android/iOS plugin compatibility or platform-setup check (1); A practical fallback that keeps report submission available (1).

Accept a supported alternative such as selecting an existing image if that feature is available and permitted, or attaching the photo later. A request to crash or require camera access for an optional attachment earns no fallback mark.

Revision sources: L09 PDF p48; L10 PDF pp34-35; Lab10 Task11.

Question 24 A validated Flutter form (5 marks)

Each correctly completed blank carries 0.5 marks.

Blank	Required completion	Marks
1	FormState	0.5
2	TextEditingController	0.5
3	_formKey	0.5
4	controller	0.5
5	validator	0.5
6	isEmpty	0.5
7	currentState	0.5
8	validate	0.5
9	trim	0.5
10	dispose	0.5

The validator returns an error string for blank input and null for valid input. Calling validate() runs the form validators before submit is called. trim() removes surrounding spaces; dispose() releases the text controller.

Answer technique: Submit only the ten numbered answers. Include no parentheses when the blank is followed by parentheses in the given code.

Revision sources: L09 PDF p42; L10 PDF p30; Lab10 Task05.

Question 25 Creating a report through a REST API (5 marks)

Each correctly completed blank carries 0.5 marks.

Blank	Required completion	Marks
1	await	0.5
2	post	0.5
3	parse	0.5
4	Content-Type	0.5
5	application/json	0.5
6	jsonEncode	0.5
7	statusCode	0.5
8	201	0.5
9	jsonDecode	0.5
10	int	0.5

HTTP header names are case-insensitive, so content-type is also accepted for [4]. The code checks the stated create status before decoding the success body. Errors are propagated to the caller as required.

Revision sources: L09 PDF p26; L10 PDF pp41,47; Lab10 Task09.

Section E Hosting selection and engineering practice

Section total 10 marks

Question 26 Hosting fit and deployment diagnosis (5 marks)

26.1 Hosting capability 2 marks

Model answer: use A for the ASP.NET Core API because it can run the required .NET process/container. Use D for the database because it provides managed PostgreSQL persistence. S cannot supply either missing runtime.

Mark allocation: API service A (0.5), linked to the .NET/runtime requirement (0.5) (1); Database service D (0.5), linked to PostgreSQL persistence (0.5) (1).

26.2 Targeted deployment fixes 3 marks

(i) Add the exact deployed dashboard origin to the API's CORS allow-list, ensure the policy is applied, and redeploy the API. (ii) Apply the pending EF Core migrations to the configured deployed PostgreSQL database through a controlled deployment step; verify the connection targets that database. (iii) Expose/use an HTTPS API endpoint and update the frontend API URL, then rebuild/redeploy the dashboard if its URL is embedded at build time.

Mark allocation: Correct CORS allow-list/policy fix for the deployed dashboard (1); Apply migrations to the intended deployed database (1); Use an HTTPS API URL to resolve mixed content (1).

The stated missing-table cause is unapplied migrations; changing a password alone does not fix it. Disabling browser security is not an acceptable mixed-content fix.

Answer technique: A deployment diagnosis should link the observed symptom to one relevant configuration or schema fix. Listing unrelated hosting platforms does not answer the question.

Revision sources: Lab11 Tasks02-05, PDF pp4-8.

Question 27 Resolving a conflict and controlling a release (5 marks)

27.1 Required merged behaviour 1 mark

```
return title.trim().toUpperCase();
```

Award 1 for preserving both operations with all conflict markers removed. An equivalent chain with both operations is accepted; retaining only one operation earns 0.5.

27.2 Complete the merge 2 marks

```
git add src/formatTicketTitle.js
git commit -m "fix: trim and uppercase ticket titles"
```

Mark allocation: Stages the resolved file; git add . is acceptable if it stages no unwanted files (1); Completes the pending merge using git commit with a meaningful message (1).

git merge --continue is an accepted alternative for completing the pending merge after staging if a suitable message is supplied through the editor. A push alone does not complete an unresolved/pending merge.

27.3 Release gate 2 marks

Model answer: hold the release, investigate and fix the failing API tests, and rerun the required checks; frontend success does not establish backend correctness. Preparing the release automatically but retaining a human production decision is Continuous Delivery.

Mark allocation: Hold/fix/rerun rather than bypass a required failed test gate (1); Continuous Delivery (1).

Revision sources: L08 PDF pp40-44,50,52,57; Lab08 Tasks03 and 05.

Section F Agentic software development

Section total 15 marks

Question 28 Reading an agent tool trace (5 marks)

28.1 Agent phases 3 marks

Reasoning or decision-making chooses what evidence or operation is needed, such as selecting the R101 lookup and then the priority-count tool. Action is the actual tool invocation, such as calling list_open_tickets with room="R101". Observation is the tool result returned to the agent, such as the two tickets or {"count":1}; it informs the next decision or final answer.

Mark allocation: Decision-making role with a concrete trace example (1); Action/tool-call role with a concrete trace example (1); Observation/tool-result role with a concrete trace example (1).

For each phase, award 0.5 for a correct explanation and 0.5 for its trace example. Do not require disclosure of private chain-of-thought; the recorded tool-selection events suffice.

28.2 Supported final answer 1 mark

Model answer: "Room R101 has 1 Open ticket with priority at least 4."

Mark allocation: Correct count of 1 (0.5); Correct scope: Open tickets in R101 with the stated priority threshold (0.5).

28.3 Missing observation 1 mark

Model answer: a timeout is not a count, so do not claim a successful count from that failed call. Retry within a limit, or report that the count could not be confirmed. Because the first tool result is complete, independently counting those verified tickets and transparently stating that recovery is also acceptable.

Mark allocation: Distinguishes a failed call from evidence (0.5) and gives a sound bounded recovery or uncertainty response (0.5) (1).

Revision sources: L05 PDF pp29-31; Lab05 Task02.

Question 29 Grounded answers and retrieval evaluation (5 marks)

29.1 Grounded response 2 marks

Model answer: "Keep clear of the affected area and telephone campus security on extension 202 [Safety Guide page 4]. This passage gives no repair-time estimate, so I cannot confirm how many minutes the repair will take; ask campus security or the maintenance team for an update."

Mark allocation: Accurately conveys both supplied safety actions with the supporting citation (1); Clearly states the repair-time information is unavailable and does not invent an estimate (1).

The extension and passage are fictitious practice data. A response can use different wording; it must remain within the evidence. A citation does not earn a mark if it is attached to an unsupported repair-time claim.

29.2 Retrieval diagnosis 3 marks

(i) The failure starts in retrieval: the only relevant passage is outside the three passages supplied to the model. (ii) Recall@3 = 0 / 1 = 0, because zero relevant passages were retrieved and one passage is known to be relevant. (iii) For this ranked list, raise k to at least 8 so the known passage is included, within the context budget; alternatively improve reranking so the relevant passage moves into the top 3.

Mark allocation: Identifies the retrieval-stage omission (1); Correct relevant counts and Recall@3 value (1); A targeted change that can include the known passage (1).

Accept a justified retriever/reranker improvement. Retraining only the generator does not address the missing passage. Increasing k may add irrelevant context, so it should be evaluated rather than assumed to improve every query.

Revision sources: L06 PDF pp7,22,25,43; Lab06 Part01; Lab07 Part04.

Question 30 Agent handoffs and recoverable approval (5 marks)

30.1 Structured handoff 2 marks

```
{
  "task": "Propose repair steps for the reported R101 fault",
  "ticket_id": 82,
  "evidence_refs": ["evidence/R101_report.md"],
  "expected_output": "Repair plan with evidence references"
}
```

Mark allocation: Useful task describing the planner's work (0.5); ticket_id identifies ticket 82 (0.5); evidence_refs points to the saved evidence (0.5); expected_output specifies repair steps with source references (0.5).

Labelled lines are equally acceptable. Do not insist on JSON punctuation if the structured information is clear.

30.2 Evidence references 1 mark

Model answer: a short evidence reference avoids repeating a large transcript in every prompt, reducing token use and distraction while allowing the planner to load the relevant evidence. Accept one clear benefit such as lower context cost, focused input or easier audit/recovery.

Mark allocation: One correctly explained benefit of saved references (1).

30.3 Persistent pause and resume 2 marks

Model answer: use a durable database/file-backed checkpointer to persist the pending state and its thread/run identity before returning the approval request. After restart, reload that same authorized thread and resume it with the explicit approval or denial; execute the proposed action only after approval, and cancel or revise it after denial.

Mark allocation: Durable persisted checkpoint, rather than only in-memory state (1); Restore the same pending workflow and resume using the actual approval/denial decision (1).

Accept durable SQLite/PostgreSQL or another suitable persistent checkpoint store. Merely restarting the whole request without restoring its pending state does not demonstrate resume.

Revision sources: L07 PDF pp19-20; Lab06 Part05 and Part06, PDF pp19-22.